

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

Sector 62, Noida

Department of Computer Science & Engineering



Performance Engineering Lab (17M15CS122)

PROJECT REPORT

Title

Network Protocol Performance Analyzer

Team Members

Abhijeet Kumar	22803029
Viyom Shukla	22803030
Devang Dixit	22803031

Submitted To:

Dr. Amitesh

Department of CSE, JIIT Sector 62, Noida.

CHAPTER 1

INTRODUCTION

1.1 What This Project Is About

We picked this topic because queues show up everywhere in networking, and none of us had ever actually seen one behave. We had only solved problems about them on paper. The story itself is simple enough: a router has a buffer, traffic comes in faster than the link can drain it, the buffer fills, delay goes up, and after that packets start getting dropped.

What we built is a Python tool that plays this out packet by packet, then checks its own numbers against the queuing theory formulas. It also runs TCP and UDP over the same overloaded link to see how each one copes. If the simulation and the maths agree, both are probably right. They agree.

1.2 The Problem

Doing this on real hardware was never really an option for us, and even labs that have the hardware struggle to repeat an experiment exactly. The formula route has its own problem. Little's Law will happily give you the average delay, but it will not show you a buffer overflowing at double load. A simulator sits in the middle. It behaves like the real thing at packet level, and since the $M/M/1$ maths is exact, we can always check whether our simulator is lying to us.

1.3 Objectives

- Simulate $M/M/1$, $M/M/c$ and finite-buffer $M/M/c/K$ queues.
- Check the simulation against Little's Law and Erlang-C.
- Measure throughput, delay and packet loss at different loads.
- Compare TCP and UDP as traffic grows.
- Figure out which setup handles a given load best.

1.4 Scope

This is a lab project, so the audience is mostly us and whoever takes this course next. We still put real effort into making it easy to run. There is a web dashboard with sliders, a command-line menu, and a Colab notebook — that last one needs zero setup, it just runs in the browser. Every graph in this report can be reproduced in a few minutes.

CHAPTER 2

BACKGROUND

2.1 Queuing Theory, Quickly

The notation takes a minute to get used to, but the idea underneath is ordinary. M/M/1: arrivals are Poisson, service times are exponential, one server. M/M/c is the same thing with c servers. M/M/c/K adds a hard cap of K packets in the system, and that cap is what creates packet loss — a packet arriving at a full system has nowhere to go. We lean on two exact results. Little's Law, $L = \lambda \cdot W$. And Erlang-C, which gives the probability that an arriving packet finds every server busy. The fact that both are exact is the whole reason validation works. There is no error bar to hide behind.

2.2 Discrete-Event Simulation

SimPy does the actual simulation work. It is event based, so instead of ticking time forward it jumps: arrival, service start, departure, next arrival. That sounds like an implementation detail. It is actually why the whole thing is fast — thousands of simulated seconds finish in a moment — and why it can drop packets from a full buffer the same way a router does, which no formula shows you directly.

2.3 Tools

Technology	Purpose
Python 3.x	Core language
SimPy	Discrete-event queue simulation
NumPy	Random arrival and service times
SciPy / math	The theory formulas
Pandas	Per-packet logs and tables
Matplotlib	All graphs
Streamlit	Web dashboard

Table 2.1: What we used and why

CHAPTER 3

REQUIREMENTS

3.1 What It Has To Do

- Generate Poisson arrivals at any chosen rate.
- Simulate all three queue models and drop packets when the buffer is full.
- Compute throughput, delay, loss and utilisation, and compare each with theory.
- Sweep TCP vs UDP across rising load and plot everything.

3.2 How Well It Has To Do It

- Accuracy: within a few percent of theory.
- Reproducibility: same seed, same result, every time.
- Simplicity: sliders and menus, runs anywhere including the browser.

3.3 System Requirements

Category	Requirement
Processor / RAM	Intel Core i3 or above, 4 GB RAM
Operating System	Windows / Linux / macOS
Language	Python 3.x
Libraries	SimPy, NumPy, SciPy, Pandas, Matplotlib, Streamlit
Tools	VS Code, Git and GitHub

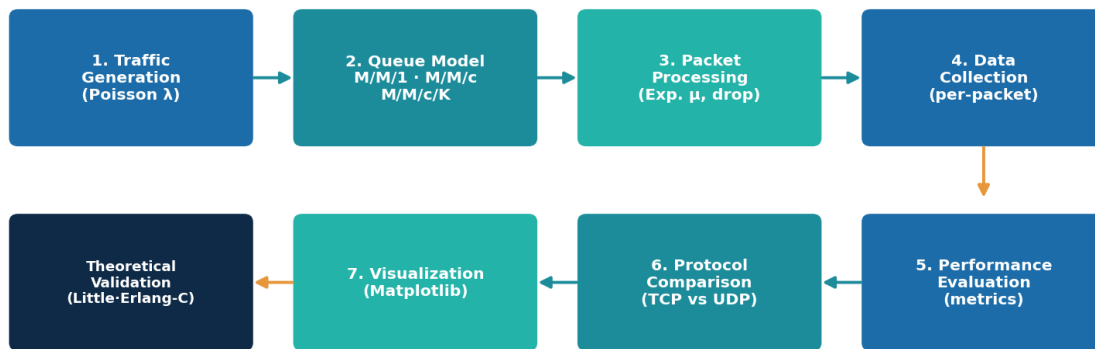
Table 3.1: Hardware and software

CHAPTER 4

DESIGN AND IMPLEMENTATION

4.1 Architecture

We kept the code split into small files on purpose, mostly because it made debugging bearable. Each module does one job and hands plain data to the next one, so the simulation, the maths and the plotting never get tangled up in each other.



Modular Python pipeline — SimPy discrete-event simulation validated against queuing theory

Figure 4.1: The seven-stage pipeline

Module	Job
traffic_generator.py	Poisson arrivals, exponential service times
queue_models.py	SimPy simulator for all three models
theoretical.py	Little's Law, Erlang-C, M/M/c/K solution
metrics.py	Metrics + the simulation-vs-theory table
protocol_comparison.py	TCP vs UDP under load
visualization.py	All graphs

Table 4.1: Modules

4.2 How the Engine Works

The engine itself is short. A SimPy resource with capacity c is the server pool. The arrival loop draws an exponential gap, creates a packet, checks occupancy — and if the system already holds K packets, the new one is marked dropped and that is it. That is the entire buffer overflow model. Every served packet logs four things: when it arrived, how long it waited, its total time in the system, and the fact that it made it out. Pandas turns the log into a table and the metrics come from a handful of lines on top of that. One thing we insisted on early was seeded randomness. Run any scenario twice and you get identical output, down to the last decimal.

4.3 Deployment

The repo is public on GitHub. Streamlit Community Cloud watches the main branch and rebuilds the dashboard whenever we push, which saved us a lot of works-on-my-machine arguments. The live app is at network-performance-analyzer.streamlit.app, and the Colab notebook opens straight from a link in the README. Nothing to install anywhere.

CHAPTER 5

TESTING AND RESULTS

5.1 Test Cases

Test Case	Expected	Status
M/M/1 vs theory	Match within ~2%	Pass
M/M/c vs theory	Matches Erlang-C	Pass
M/M/c/K finite buffer	Loss matches blocking probability	Pass
TCP vs UDP sweep	Expected trade-off appears	Pass
Dashboard and notebook	Run end to end	Pass

Table 5.1: Tests and outcomes

5.2 Simulation vs Theory

Every run prints its measured numbers next to the theory. Here are five of them, straight from real runs, not cherry picked. The M/M/2/6 loss line was the one we were most nervous about, and it came out fine.

Scenario and Metric	Simulation	Theory	Error
M/M/1 - Utilisation	0.805	0.800	0.005
M/M/1 - Avg delay W (s)	0.512	0.500	0.012
M/M/2 - Avg delay W (s)	0.231	0.229	0.002
M/M/2/6 - Packet loss	0.110	0.113	0.003
M/M/2/6 - Throughput (pkt/s)	16.10	15.97	0.13

Table 5.2: Measured vs theoretical

5.3 Graphs

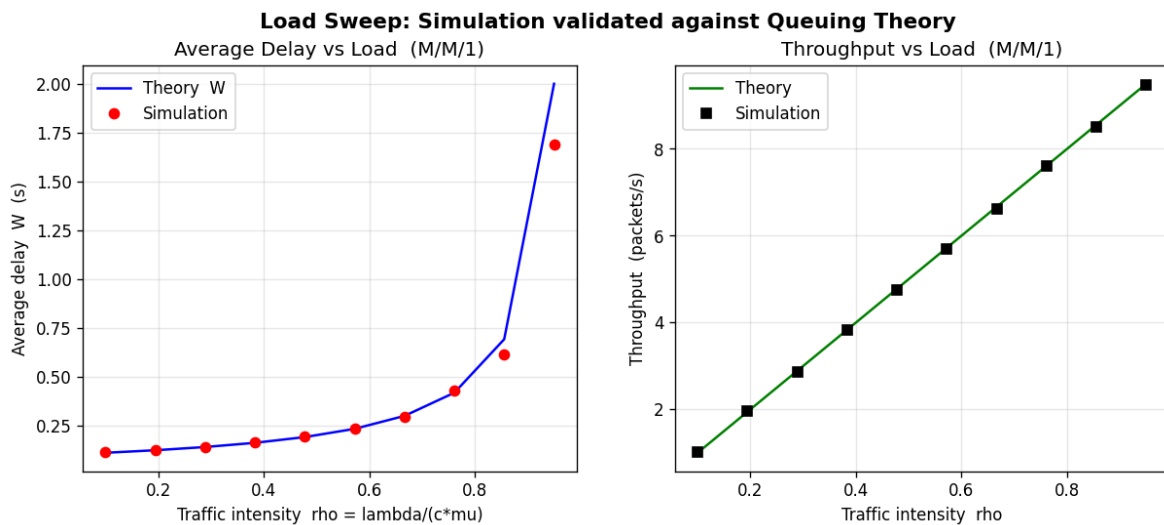


Figure 5.1: M/M/1 load sweep. Dots are simulation, lines are theory. Delay explodes as ρ nears 1.

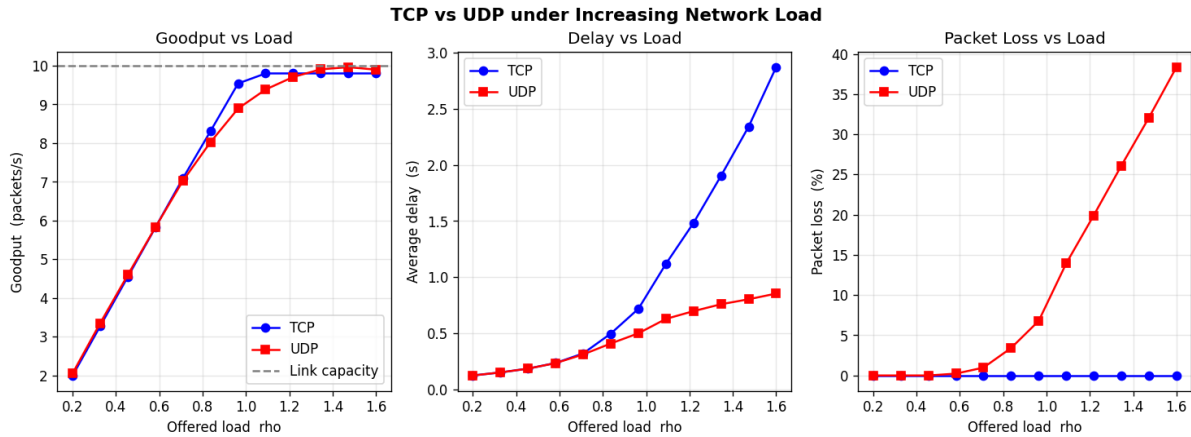


Figure 5.2: TCP vs UDP — goodput, delay and loss as load rises.

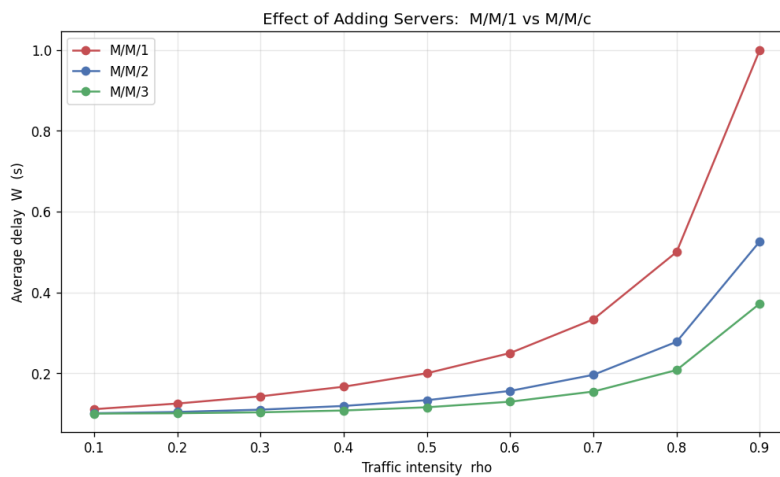


Figure 5.3: More servers, less delay at the same per-server load.

5.4 What We Found

- Agreement with theory was 1-2% everywhere, sometimes better.
- The delay curve has a knee. Flat, flat, flat, then almost straight up once ρ gets near 1.
- Going from one server to two at the same per-server load cut delay by more than half. A third server helped less. Diminishing returns, basically.
- The overload test was the fun one. UDP kept its delay low and quietly lost up to 38% of everything we sent. TCP delivered every single packet, but its delay kept climbing the whole time.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The plan was to connect the formulas from class to something we could actually run, and that part worked out. The numbers match. Close enough that whenever they did not match early on, the bug turned out to be in our code, never in the theory — which is a lesson in itself. If we had to pick one takeaway it would be the TCP versus UDP graph. Reliability is not free. You pay for it in delay, and now we have measured exactly how much. Everything in this report can be re-run in a few minutes from the web app, the command line or the Colab notebook.

6.2 Future Scope

- Non-Markovian models (M/G/1) and bursty traffic.
- Priority queues and scheduling policies.
- A more detailed TCP congestion-control model.
- Chaining queues to model a full network path.

REFERENCES

- [1] R. Jain, The Art of Computer Systems Performance Analysis. Wiley, 1991.
- [2] L. Kleinrock, Queueing Systems, Volume 1: Theory. Wiley-Interscience, 1975.
- [3] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Pearson, 2011.
- [4] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 7th ed. Pearson, 2017.
- [5] "SimPy Documentation." [Online]. Available: <https://simpy.readthedocs.io>
- [6] "NumPy and SciPy Documentation." [Online]. Available: <https://numpy.org>